

Title: E-COMMERCE FLASH SALE

Problem Statement: where 10,000 concurrent users compete for only 100 units of inventory.

Goal: You must design a scalable system for this scenario without **overselling, duplication transactions, or dropping order state under service failure**

Focus on: system design, architecture trade-offs, and failure recovery-not volume of code

1. Requirement Analysis

Functional Requirements(FR) – what a system must do

FR describes specific behaviors, features, and business rules of a system. If FR is missing, the core system breaks or fails to complete its task.

Focus: System features, workflows and user actions

Structure: Often written using a verb, a noun and when (e.g., “the user can reset a password”)

- A user can log in using an email address and password.
- The system calculates total shipping costs and taxes at checkout.
- A user receives an email notification after completing a payment.

1.Product & Cart Management:

Allow Product discovery, cart operations(Add, delete, read the cart), and “Buy Now” checkout workflows.

Checkout – the step where the customer reviews the order, enters delivery/payment details, and places the order.

2.Inventory Reservation:

Temporarily reserve inventory upon checkout with auto-release on payment expiry or failure

Meaning: When a customer starts checkout(enters the checkout workflow)→temporarily keep those products aside for them→If they don't complete payment, make the products available again.

3.Payment Processing:

Safe, **idempotent integration** with payment gateways handling success, failure, timeout, and retries.

Idempotency = repeating the same request should NOT create duplicate results

Customer clicks “Pay” → ₹500 is charged.

Due to a network problem, the request is send again.

Without idempotency: ₹500+ ₹500= ₹1000 charged

With idempotency: First request→ ₹500 charged. Second identical request→system recognizes it was already processed→no second charged

Idempotency is implemented using a **unique_idempotency_key** for each payment request.

Payment_id = PAY1@3

If the same “PAY1@3” request comes again, the payment gateway says “already processed” instead of charging again

The **payment gateway stores the idempotency key** after processing the first request.

Example:

1. Your app sends PAY123 → gateway charges ₹500 → **stores PAY123**.
2. Your app sends PAY123 again → gateway checks its stored keys.

3. PAY123 already exists → **doesn't charge again; returns the previous result.**

4. Order lifecycle Management:

State machine transitions from

CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED → DELIVERED

5. Notification & Delivery:

Dispatch confirmation events and coordinate shipment tracking.

Non-Functional Requirements (FR) – define how well the system performs those actions ('Scalability, Availability, Consistency, Security, etc')

Non-FR describe quality attributes, standards, and operational constraints. They dictate how the system runs rather than specific behaviors. If a NFR is missing, the system might still function, but it may be too slow, insecure, or hard to use.

Focus: Performance, security, usability, and reliability

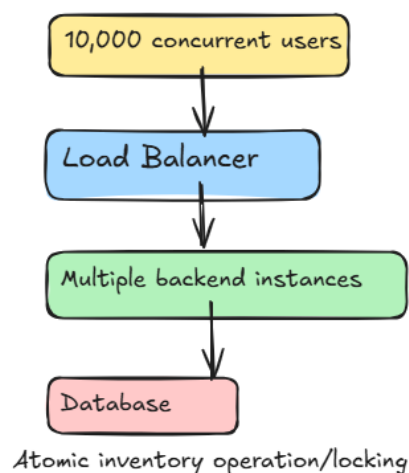
Structure: Often measured using specific metrics or benchmarks (e.g., "load time under 2 seconds")

- Security: All user passwords and personal data must be encrypted.
- Scalability: The platform must support 10,000 active users at the same time.
- Reliability: The system must maintain 99.9% uptime.
- Usability: The interface must follow standard accessibility rules for easy navigation.
- Performance: The system must load web pages in under 2 seconds.

1. Concurrency & Throughput:

Support 10,000 concurrent purchase requests initially, with horizontal scaling strategies capable of addressing up to 500,000 req/sec

Horizontal scaling doesn't specifically mean microservices. It means adding more instances of whatever component is becoming a bottleneck.



Concurrent inventory reservation without overselling

2.Consistency & Zero Overselling:

Strict consistency for inventory—100 items must lead to **at most 100** successful sales (no negative stock).

The key is to make **checking stock + decreasing stock one atomic operation**

Suppose DB has: stock = 100

When a purchase request arrives:

```
UPDATE inventory
SET stock = stock - 1
WHERE product_id = 123
AND stock > 0;
```

Implement **Atomic Stock Update**. The 10,000 requests can arrive concurrently, but the database's atomic operation ensures only 100 succeed.

With concurrent requests, the database decides the order in which transactions acquire the lock/execute the atomic update.

Why chose “Atomic Stock Update” instead of “queue + sequence number” architecture?

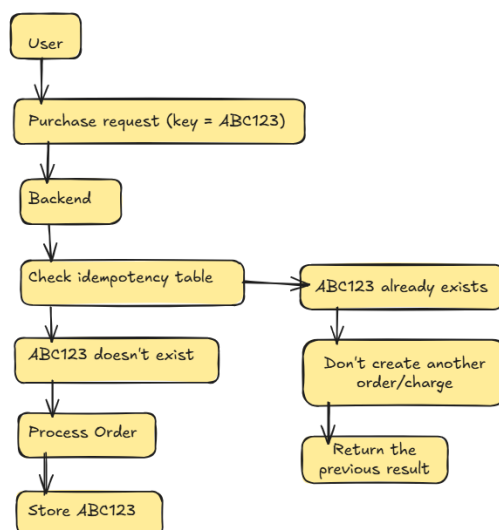
Because the requirement is “at most 100 successful sales”, not “first 100 users must win”. Atomic Stock Update → is simple, fast, guarantees no overselling. Whereas “Queue + sequence number” → is needed only if we must guarantee strict arrival order/first-come-first serve.

Also Queuing adds extra complexity and latency without providing a required benefit.

Architecture choice: Start with **atomic DB update**. Add a queue if traffic becomes so high that the database cannot handle the burst efficiently.

3.Idempotency & Reliability:

Guarantee duplicate purchase/payment requests do not produce duplicate charges or orders.



Think of Idempotency key as a unique receipt number for one purchase attempt.

1. Generate a unique key

When the user clicks **Buy**, your backend creates:

idempotency_key = "PURCHASE_ABC123"

That same key belongs to that purchase attempt.

2. Database prevents duplicate orders

Your orders table has a **unique constraint** on that key:

Idempotency_key	order_id
PURCHASE_ABC123	ORDER_001

First request:

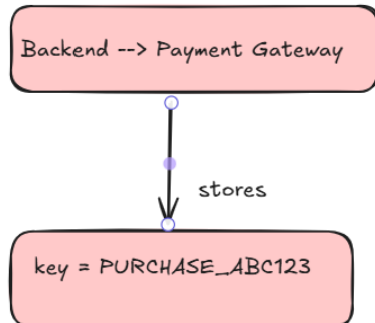
PURCHASE_ABC123 → doesn't exist, creates ORDER_001, stores the key

Same request comes again,

PURCHASE_ABC123 → already exists, don't create another order, return ORDER_001

3. Payment gateway does the same thing

You send the same key to the payment gateway



If the payment request is accidentally send twice, the gateway sees:

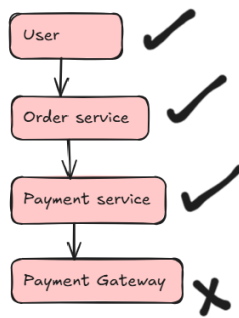
PURCHASE_ABC123 → already processed

So it doesn't change the customer twice

4. High Availability & Fault Tolerance: System should remain available and handle failures or delays in dependent services without crashing or causing incorrect transactions. (e.g., Order Service, Payment Gateways)

If another service becomes slow or unavailable, our system should still keep working as much as possible instead of completely failing

Don't let one failed dependency bring down the whole system



Important: You should not say the purchase succeeded when payment hasn't been confirmed. The system remains available, but the transaction remain safely pending. Can implement it with:

- ☐ **Timeouts** — Don't wait forever for the payment gateway.
- ☐ **Retries** — Retry temporary failures a few times.
- ☐ **Circuit breaker** — If the gateway keeps failing, temporarily stop calling it.
- ☐ **Pending status** — Keep the order as `PAYMENT_PENDING` instead of losing it.
- ☐ **Queue** — Store failed/pending payment work and process it later.
- ☐ **Fallback response** — Tell the user: *"Payment is being processed; we'll update you."*

Payment Gateway fails

↓
Timeout / retry
↓
Still failing?
↓
Order = `PAYMENT_PENDING`
↓
Queue → retry later

4. **Low Latency:** Fast response times at the API Gateway via edge caching, request throttling, and queueing mechanisms
CDN caching. Content is cached near users so they don't have to reach your main server. Request throttling limits how many requests a client can send within a time period

Assumptions:

1. Trying to buy $\neq$ having the product reserved

If a user keep retrying but never successfully reserve an item, they should not get the product. Otherwise, someone could spam requests and unfairly hold/obtain inventory

Rate limiter $\rightarrow$ prevents one user from flooding requests

The page can show: "Item reserved for you, complete payment within 10 minutes"

Until they get that reservation, they're simply waiting/trying. Not entitled to the product.

